

Đề Xuất Các Bài Thực Hành và Dự Án (Proposal for Labs and Projects)

Contents (Mục lục)

- Kịch Bản và Yêu Cầu: Hệ Thống Quản lý Nhân sự (HRM System)
- Các Loại Kiến Trúc Phần mềm (Để So sánh và Sử dụng trong Lab)
- 8 Bài Thực Hành (Lab)
- Các Dự Án Khác (Cơ bản đến Trung bình)

Hệ Thống Quản lý Nhân sự (HRM): Kịch Bản và Yêu Cầu

Kịch Bản: Nền Tảng HRM "Microservices-Oriented" Hệ thống HRM là một nền tảng quản trị nhân sự nội bộ giàu tính năng, được thiết kế để số hóa toàn bộ vòng đời nhân sự (định danh, chấm công, nghỉ phép, tính lương). Nền tảng đòi hỏi tính chính xác tuyệt đối về dữ liệu tài chính (lương), khả năng sẵn sàng cao (High Availability), và trải nghiệm thời gian thực (Real-time). Đặc biệt, hệ thống phải đáp ứng tốt khả năng mở rộng khi quy mô công ty lên tới hàng ngàn nhân viên thông qua kiến trúc phân tán Microservices.

Yêu Cầu Chính

Loại	Mô Tả Yêu Cầu	Động Lực Kiến Trúc (Architectural Driver)
Chức năng	Quản lý Định danh (Auth): Đăng ký OTP, Đăng nhập JWT, Phân quyền RBAC (SUPERADMIN, MANAGER, STAFF), Quản lý phòng ban.	Bảo mật, Quản lý Định danh phân tán.
Chức năng	Quản lý Nghỉ phép (Leave): Tạo đơn, Duyệt đơn theo cấp bậc, Tính toán quỹ phép, Hiển thị ma trận lịch làm việc.	Hiệu suất xử lý logic, Tính toán vẹn dữ liệu.
Chức năng	Chấm công & Tính lương (Salary/Attendance): Ghi nhận giờ vào/ra, tự động tính công chuẩn, phạt đi muộn/không phép, chốt lương theo tháng.	Xử lý dữ liệu lớn (Batch Processing), Tính chính xác.

Chức năng	Truyền thông nội bộ (Noti): Gửi thông báo hệ thống, Cập nhật trạng thái realtime khi có đơn duyệt/đơn mới.	Tích hợp không đồng bộ (Asynchronous).
Phi Chức năng	Hiệu suất & Real-time: Mọi thay đổi trạng thái (duyet đơn) phải hiện Toast notification tức thì trên màn hình Admin/Manager.	Kiến trúc Hướng Sự kiện (EDA), Pub/Sub.
Phi Chức năng	Khả năng Sẵn sàng & Chịu lỗi: Database phải sống sót ngay cả khi 1 node sập. Hệ thống không bị treo nếu dịch vụ Gửi Email lỗi.	Dự phòng (Postgres HA Patroni, Mongo Replica Set).
Phi Chức năng	Khả năng mở rộng: Tách biệt rõ ràng các miền nghiệp vụ để có thể scale riêng lẻ dịch vụ Tính lương vào cuối tháng.	Phân rã Microservices, Cân bằng tải (Kong/Nginx).

Các Loại Kiến Trúc Phần mềm (Để So sánh và Sử dụng trong Lab)

Hệ thống HRM là một nghiên cứu điển hình tuyệt vời để so sánh các phương pháp kiến trúc khác nhau.

1. Kiến Trúc Phân Tầng (Layered Architecture - Monolithic):

- **Mô tả:** Ban đầu, hệ thống có thể được viết chung trong một thư mục backend duy nhất với các tầng: Router (API) -> Controller -> Service -> Repository -> PostgreSQL.
- **Trường hợp sử dụng:** Lý tưởng cho giai đoạn MVP (Minimum Viable Product) để kiểm chứng nghiệp vụ HRM nhanh chóng.
- **Ưu điểm/Nhược điểm:** Dễ phát triển ban đầu, nhưng khi module Tính Lương chạy nặng sẽ làm "treo" luôn module Xin Nghỉ Phép.

2. Kiến Trúc Microservices (Đang áp dụng):

- **Mô tả:** Ứng dụng được chia thành auth_service, leave_service, salary_service và noti_service độc lập. Dùng Kong/Nginx làm API Gateway.
- **Trường hợp sử dụng:** Môi trường doanh nghiệp vừa và lớn, nơi team IT chia thành các squad nhỏ quản lý từng service.
- **Ưu điểm:** Khả năng mở rộng cục bộ (Scale riêng salary_service), sử dụng đa cơ sở dữ liệu (PostgreSQL cho dữ liệu lõi, MongoDB cho Logs).

3. Kiến Trúc Hướng Sự Kiện (Event-Driven Architecture - EDA):

- **Mô tả:** Giao tiếp qua Message Broker (Redis). Khi có Đơn nghỉ phép mới (leave_service), nó sẽ "phát" (Publish) một sự kiện vào Redis, và noti_service sẽ "lắng nghe" (Subscribe) để gửi Socket.io.
- **Trường hợp sử dụng:** Các tác vụ chạy ngầm như Gửi Email (NodeMailer) hoặc tính lương hàng loạt.
- **Ưu điểm:** Phản hồi UI tức thì (Non-blocking), dịch vụ Noti sập thì Leave vẫn lưu database bình thường.

8 Bài Thực Hành (Lab)

8 bài thực hành sau đây được thiết kế sát với mã nguồn dự án HRM, dẫn dắt từ thiết kế ban đầu đến triển khai Microservices và đánh giá chất lượng.

Lab	Tiêu đề	Trọng tâm Kiến trúc	Hoạt động & Hướng dẫn Từng bước Chi tiết
1	Thu thập Yêu cầu & Mô hình hóa (Use Case)	N/A (Tiền Kiến trúc)	Hoạt động: Xác định Yêu cầu đối với hệ thống HRM. 1. Xác định tác nhân: STAFF, MANAGER, SUPERADMIN. 2. Xác định Use cases: Chấm công, Duyệt phép, Chốt lương. 3. Chi tiết hóa use case "Tính lương cuối tháng". 4. Xác định Yêu cầu quan trọng (ASR): Phải xử lý đồng thời 1000 nhân viên mà không treo hệ thống.
2	Thiết kế Kiến trúc Phân Tầng (Logical View)	Phân Tầng (Bên trong Service)	Hoạt động: Thiết kế kiến trúc bên trong leave_service. 1. Chia logic trong file leaves.js hiện tại thành 3 tầng: LeaveRouter, LeaveService và LeaveRepository. 2. Vẽ biểu đồ Component UML luồng đi từ API request -> Nghiệp vụ (Kiểm tra quỹ phép) -> Persistence (Lưu PostgreSQL).
3	Triển khai Kiến trúc Phân Tầng (CRUD)	Phân Tầng	Hoạt động: Tái cấu trúc file leaves.js . 1. Tách các câu SQL (như INSERT INTO leave_requests) ra khỏi Router. 2. Đưa logic kiểm tra giới hạn (tối đa 5 người nghỉ/ngày, không nghỉ quá 3 ngày) vào lớp Business Logic. 3. Triển khai API POST /api/leaves chuẩn 3 lớp.
4	Phân rã Microser	Microservices	Hoạt động: Phân rã Monolith và thiết lập API Contract. 1. Xác định ranh giới cho auth_service và leave_service. 2.

	vices & Giao tiếp		Xác định cách salary_service lấy dữ liệu chấm công từ DB (Thông qua API nội bộ thay vì query trực tiếp). 3. Vẽ mô hình C4 Context Diagram kết nối với NodeMailer.
5	Triển khai Dịch vụ Salary Service	Microservices	Hoạt động: Phân tích logic file payroll.js . 1. Tách biệt hoàn toàn salary_service ra cổng riêng (port 3000). 2. Chạy container độc lập. 3. Thực thi hàm syncPayrollRealtime độc lập để kết nối với cơ sở dữ liệu và tính toán lương hàng tháng.
6	Triển khai API Gateway (Kong/Nginx)	Microservices	Hoạt động: Cấu hình Gateway từ file docker-compose.yml . 1. Cấu hình Nginx port 80. 2. Chuyển tiếp /api/auth sang auth_service và /api/leaves sang leave_service. 3. Tìm hiểu cách Kong API Gateway quản lý Rate Limiting và định tuyến thay cho Nginx.
7	Kiến trúc Hướng Sự kiện (EDA) với Redis	Hướng Sự kiện	Hoạt động: Triển khai luồng Real-time Pub/Sub. 1. Mở Redis container. 2. Khi leaves.js gọi POST /, thay vì emit Socket trực tiếp (gây khó mở rộng), hãy viết logic Publish message "NEW_LEAVE" vào Redis. 3. Dịch vụ noti_service Subscribe Redis và thực hiện bắn io.emit tới trình duyệt.
8	Tầm nhìn Triển khai & Phân tích ATAM	Triển khai & Đánh giá	Hoạt động: Vẽ biểu đồ vật lý từ docker-compose.yml . 1. Vẽ các node: Nginx, Kong, 4 Node.js Services, Postgres (Spilo/Patroni HA), Mongo Replica Set (rs0). 2. Đánh giá ATAM: Kịch bản nếu postgres-1 sập, hệ thống HAProxy chuyển sang postgres-2 ra sao? Kịch bản khi cần Scale salary_service cuối tháng.

Các Dự Án Khác

Dưới đây là 4 dự án phụ trợ từ cơ bản đến trung bình, có thể dùng để luyện tập kỹ năng trước khi làm đồ án lớn:

Dự án 1: Ứng Dụng Danh Sách Việc Cần Làm (To-Do List) (Cơ bản)

- **Kịch bản:** Công cụ cá nhân quản lý công việc. Thêm, xem, đánh dấu hoàn thành, xóa (CRUD).
- **Kiến trúc:** 1-Tầng (Client-Server đơn giản).

- **Các bước Lập trình (Node.js/Express):**

- Tạo mảng in-memory: `let tasks = [{ id: 1, title: "Họp", done: false }]`.
- Viết API POST `/tasks` để push vào mảng. GET `/tasks` trả về danh sách.

Dự án 2: Hệ Thống Quản lý Điểm Sinh viên (Trung bình)

- **Kịch bản:** Quản trị viên thêm Khóa học, Sinh viên. Giảng viên nhập điểm.
- **Kiến trúc:** Phân tầng (2-Tầng) với SQL.
- **Các bước Lập trình (PostgreSQL & Node.js):**
 - Lược đồ SQL: 3 bảng Students, Courses, Enrollments.
 - Logic: Viết hàm SQL JOIN hoặc API tính điểm trung bình (GPA) có trọng số.

Dự án 3: Dịch Vụ Rút Gọn URL (Trung bình)

- **Kịch bản:** Giống Bit.ly. Biến URL dài thành mã ngắn (VD: `hr.m/ab12`), đếm số lượt click.
- **Kiến trúc:** 3-Tầng tập trung vào hiệu năng Đọc (Read-heavy).
- **Các bước Lập trình:**
 - Dùng Hash function hoặc Base62 chuyển ID tăng dần thành chuỗi ký tự.
 - API GET `/:code` query DB tìm URL gốc, cộng `click_count += 1`, sau đó trả về `res.redirect(301, long_url)`.

Dự án 4: Ứng Dụng Chat Thời Gian Thực (Trung bình)

- **Kịch bản:** Ứng dụng chat phòng ban. Nhắn tin hiển thị ngay không cần F5.
- **Kiến trúc:** Hướng sự kiện (WebSockets).
- **Các bước Lập trình (React + Socket.io):**
 - Backend: `io.on('connection') -> Lắng nghe socket.on('chat_msg', msg) -> Phát io.emit('chat_msg', msg)`.
 - Frontend: Dùng `useEffect` kết nối Socket và cập nhật State UI ngay lập tức.